

1-4 · Dockerfile 작성 및 이미지 최적화 · 실습 1H

이미지 최적화

베이스 교체와 멀티스테이지로 줄이고, 전후를 수치로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

- ① python:3.12 와 slim 으로 빌드한 이미지 크기를 docker images 로 비교한다
- ② RUN 을 합쳐 층수를 줄이는 이유를 말할 수 있다
- ③ 멀티스테이지 빌드로 빌드 도구를 최종 이미지에서 뺀다
- ④ docker history 로 층별 크기를 보고 1-5 에 쓸 이미지를 확정한다

크기 수치는 정리·회고 시간에 공유합니다

다섯 단계 — app 폴더에서 이어서

실험용 이미지는 마지막 단계에서 한꺼번에 정리합니다

1. 크기 측정	2. 층 합치기	3. 멀티스테이지	4. 들여다보기	5. 확정
fat 과 slim 비교	RUN 여러 번 → 한 번	builder → 실행 이미지	docker history	data-app:latest

베이스 하나로 수백 MB — fat 과 slim 을 나란히

python:3.12 — data-app:fat

sed 로 베이스 줄만 바꿔 Dockerfile.fat 으로 빌드

- 컴파일러·개발 도구가 다 들어 있다
- 0.3 보다 수백 MB 이상 크면 정상
- 이 수치를 정리·회고에서 공유

VS

python:3.12-slim — data-app:0.3

실행에 필요한 최소한 — 이번 과정의 기본값

- docker images data-app 로 SIZE 비교
- 배포 때마다 네트워크로 전송된다
- 1GB × 서버 10대 = 10GB 가 오간다

RUN 세 번은 세 층 — 설치와 정리는 한 층에서

1

나쁜 예 — RUN 세 번

apt-get update · install · rm 이 각각 한 층 — 캐시 파일이 아래층에 남는다

2

좋은 예 — && 로 한 층

update && install -y --no-install-recommends && rm -rf /var/lib/apt/lists/* 를 한 RUN 에

3

층은 지워도 사라지지 않는다

아래층 파일을 위층에서 rm 하면 화면엔 안 보여도 크기엔 그대로 — 비밀번호·키도 마찬가지

기억할 것

설치와 정리는
같은 층에서

3단계

멀티스테이지 — 앞 단계의 결과물만 옮겨 담는다



빌드할 때만 필요한 도구를 최종 이미지에 남기지 않는다 — 효과는 컴파일이 필요한 Go·Java·C++ 에서 크다

`docker build -t data-app:multi -f Dockerfile.multi .` → 정상 실행되고 fat 보다 훨씬 작으면 성공

이미지 안을 들여다보고, 1-5 에 쓸 이미지를 확정

층별 크기	<code>docker history data-app:0.3</code>
fat 과 비교	<code>docker history data-app:fat head -n 8</code>
내 파일 확인	<code>docker run --rm data-app:0.3 ls -la /app</code>
실험용 정리	<code>docker rmi data-app:bad ... data-app:multi (5개)</code>
최종 태그	<code>docker tag data-app:0.3 data-app:latest</code>
마지막 확인	<code>docker run --rm data-app:latest</code>

app 폴더에 Dockerfile 이 있어야 1-5 Compose 가 build 할 수 있다

이 1시간이 끝날 때 확인할 것

- 1 data-app:fat 이 0.3 보다 수백 MB 이상 크다
- 2 data-app:multi 가 정상 실행되고 fat 보다 훨씬 작다
- 3 docker history 에 층별 SIZE 가 보인다
- 4 ls -la /app 에 app.py·requirements.txt·Dockerfile 이 있다
- 5 태그가 0.1·0.2·0.3·latest 만 남고 latest 가 정상 출력을 낸다

지금 예제는 패키지가 하나라 slim 단독과 멀티스테이지 차이가 작다 — 「이런 방법이 있다」까지

1-4 · 실습 1H 마무리

다음은 정리·회고 (0.5H)

최적화 전후 수치를 서로 공유하고, 오늘의 지시어를 한 장으로 정리합니다

✓ 베이스 크기 차이·재빌드 시간 차이 공유

✓ Dockerfile 지시어 요약 표

✓ 자주 겪는 오류 세 가지

✓ 1-5 Docker Compose 예고

가져갈 것

docker images 로 잔 fat 과 slim 의 SIZE